

Universidade São Judas Tadeu

Andressa Emily Rabêlo Pereira 823213904
Júlia Oliveira Rocha 823214680
Lucas Marzocca 823116813
Matheus Henrique Ferreira Guimarães 823141914
Marcos Vinicius Santos 82327399
Mylena Soares Rocha 824144075
Samuel Faustino Gomes da Costa 824147380

Projeto A3 - Relatório

Gestão e Qualidade de Software

Profº Robson Calvetti

**São Paulo
2025**

Índice

1. DESCRIÇÃO DAS DEFICIÊNCIAS IDENTIFICADAS NO CÓDIGO ORIGINAL	3
1.1 VIOLAÇÕES DOS PRINCÍPIOS SOLID	3
1.1.1 <i>Single Responsibility Principle (SRP)</i>	3
1.1.2 <i>Violação do Open/Closed Principle (OCP)</i>	3
1.1.3 <i>Violação do Dependency Inversion Principle (DIP)</i>	4
1.2 VIOLAÇÕES DO CLEAN CODE	5
1.2.1 <i>Nomes de Variáveis Não Descritivos</i>	5
1.2.2 <i>Magic Numbers (Números Mágicos)</i>	5
1.2.3 <i>Código Duplicado (Violação do Princípio DRY)</i>	6
1.2.4 <i>Funções Muito Longas</i>	6
1.2.5 <i>Comentários Inadequados ou Ausentes</i>	7
1.3 PROBLEMA DE ESTRUTURA E ARQUITETURA	7
1.3.1 <i>Uso Excessivo de Variáveis Globais</i>	7
1.3.2 <i>Ausência de Orientação a Objetos</i>	8
1.3.3 <i>Acoplamento Alto Entre Componentes</i>	9
1.4 PROBLEMAS DE TRATAMENTO DE ERROS	9
1.4.1 <i>Ausência de Tratamento de Exceções</i>	9
1.4.2 <i>Verificações de Null/None Inadequadas</i>	10
1.5 PROBLEMAS DE CONSISTÊNCIA	11
1.5.1 <i>Inconsistência na Geração de IDs</i>	11
1.5.2 <i>Inconsistência na Nomenclatura</i>	11
1.6 PROBLEMAS DE PERFORMANCE	12
1.6.1 <i>Busca Linear Repetitiva</i>	12
1.6.2 <i>Salvamento Excessivo em Disco</i>	12
1.7 PROBLEMAS DE VALIDAÇÃO	13
1.7.1 <i>Validações Insuficientes</i>	13
1.7.2 <i>Validações Espalhadas</i>	14
2. JUSTIFICATIVAS PARA AS MUDANÇAS IMPLEMENTADAS	15
2.1 SEPARAÇÃO DO CÓDIGO EM MÚLTIPLOS ARQUIVOS (MODULARIZAÇÃO)	15
2.2 REMOÇÕES DE VARIÁVEIS GLOBAIS E ENCAPSULAMENTO EM ATRIBUTOS DE INSTÂNCIA	16
2.3 CRIAÇÕES DAS CLASSES DE DOMÍNIO: LIVRO, USUÁRIO E EMPRÉSTIMO	17
2.4 CENTRALIZAÇÕES DAS REGRAS DE NEGÓCIO NA CLASSE BIBLIOTECA	19
2.5 IMPLEMENTAÇÕES DO PADRÃO DE PROJETO FACTORY METHOD	20
2.6 REFATORAÇÃO DO MECANISMO DE PERSISTÊNCIA COM SERIALIZAÇÃO AUTOMÁTICA	21
2.7 ISOLAMENTOS DA INTERFACE DE USUÁRIO EM MÓDULO DEDICADO	24
2.8 IMPLEMENTAÇÕES DE SUITE COMPLETA DE TESTES UNITÁRIOS	26
3. DESCRIÇÃO DOS TESTES UNITÁRIOS IMPLEMENTADOS	27
3.1 OBJETIVOS E ESCOPO DOS TESTES	27
3.2 TECNOLOGIAS E FERRAMENTAS UTILIZADAS	28
3.2.1 <i>Framework de Testes: unittest</i>	28
3.2.2 <i>Ambiente de Integração Contínua: GitHub Actions</i>	28
3.3 ESTRUTURA E ORGANIZAÇÃO DOS TESTES	28
3.3.1 <i>Configuração de Fixtures (setUp e tearDown)</i>	28
3.3.2 <i>Calendário e Etapas de Implementação de Testes</i>	29
3.4 CASOS DE TESTE IMPLEMENTADOS	29
3.4.1 <i>Testes de Comportamento Funcional</i>	29
3.4.2 <i>Testes de Validação e Tratamento de Erros</i>	32
3.5 PIPELINE DE INTEGRAÇÃO CONTÍNUA (CI/CD)	35
3.5.1 <i>Configuração do GitHub Actions</i>	35
3.5.2 <i>Etapas do Pipeline</i>	35
3.5.3 <i>Benefícios do Pipeline Automatizado</i>	36

3.6 PRINCÍPIOS DE DESIGN E BOAS PRÁTICAS	36
3.6.1 Isolamento Completo Entre Testes	36
3.6.2 Padrão Arrange-Act-Assert (AAA)	36
3.6.3 Foco em Um Comportamento Por Teste.....	37
3.6.4 Mensagens de Erro Claras e Informativas	37
3.6.5 Dados de Teste Realistas	37
3.7 MÉTRICAS E COBERTURA DE TESTES	37
4. CONCLUSÃO SOBRE A IMPORTÂNCIA DO CLEAN CODE NA MANUTENÇÃO DE SOFTWARE	38

1. DESCRIÇÃO DAS DEFICIÊNCIAS IDENTIFICADAS NO CÓDIGO ORIGINAL

Esta seção apresenta a análise sistemática das principais deficiências identificadas no código legado do sistema de gerenciamento de biblioteca. O objetivo é evidenciar problemas estruturais, arquiteturais e de qualidade de software que comprometem manutenibilidade, extensibilidade, legibilidade, testabilidade e confiabilidade do sistema.

O código original encontra-se disponível no repositório público:

https://github.com/Julia-Olive/USJT-2025.2-GQS-ProjetoA3/blob/main/Legacy_code

1.1 Violações dos Princípios SOLID

1.1.1 Single Responsibility Principle (SRP)

O princípio da Responsabilidade Única estabelece que uma classe ou função deve possuir apenas uma razão para mudar, ou seja, deve executar uma única responsabilidade bem definida. A violação deste princípio resulta em módulos coesos inadequadamente, dificultando testes, manutenção e reutilização de código.

Localização no Código: A deficiência manifesta-se na função `adicionarLivro()` (linhas 10-34), na função `cadastrarUsuario()` (linhas 36-57) e na função `realizarEmprestimo()` (linhas 59-94).

Descrição da Falha: A função `adicionarLivro()` acumula múltiplas responsabilidades distintas em um único bloco de código:

- Validação de campos obrigatórios (linhas 12-14)
- Validação de formato do ISBN (linhas 20-22)
- Verificação de duplicidade na base de dados (linhas 16-18)
- Criação e estruturação do objeto livro (linhas 24-31)
- Persistência de dados através da chamada `salvarDados()` (linha 33)
- Exibição de mensagens ao usuário (linhas 13, 17, 21, 32)

Esta concentração de responsabilidades viola diretamente o SRP, uma vez que alterações em qualquer uma dessas operações — seja na lógica de validação, no formato de armazenamento ou na interface de usuário — exigem modificações na mesma função, aumentando o acoplamento e o risco de introdução de defeitos.

Impacto: A violação impede a testabilidade unitária isolada de cada responsabilidade, impossibilita a reutilização de lógicas específicas em outros contextos, dificulta a manutenção evolutiva do sistema e aumenta a complexidade ciclomática da função, tornando o código mais propenso a erros e menos compreensível para desenvolvedores.

1.1.2 Violação do Open/Closed Principle (OCP)

O princípio Aberto/Fechado determina que entidades de software devem estar abertas para extensão, mas fechadas para modificação. Código que viola este princípio requer alterações diretas em sua implementação para acomodar novos requisitos, contrariando a estabilidade e previsibilidade do sistema.

Localização no Código: A deficiência está presente na estrutura de dados utilizada para representar livros (linhas 24-31) e na ausência de abstrações que permitam extensibilidade do sistema.

Descrição da Falha: O sistema utiliza dicionários Python com estrutura fixa para representar livros, conforme observado nas linhas 24-31:

```
l = {  
    'id': contador_livros,  
    'titulo': titulo,  
    'autor': autor,  
    'isbn': isbn,  
    'ano': ano,  
    'disponivel': True  
}
```

Esta implementação impede a extensão do sistema para gerenciar outros tipos de itens bibliográficos (revistas, periódicos, DVDs, e-books, teses) sem modificação substancial do código existente. Não há hierarquia de classes, interfaces ou abstrações que permitam a adição de novos tipos de materiais através de herança ou composição.

Impacto: Qualquer ampliação do escopo do sistema, requisito comum em bibliotecas reais, demanda reescrita de funções existentes, violação do princípio da estabilidade de código testado, aumento do risco de regressão e impossibilidade de extensão modular do sistema por múltiplos desenvolvedores simultaneamente.

1.1.3 Violação do Dependency Inversion Principle (DIP)

O princípio da Inversão de Dependência estabelece que módulos de alto nível não devem depender de módulos de baixo nível, mas ambos devem depender de abstrações. Além disso, abstrações não devem depender de detalhes, mas detalhes devem depender de abstrações.

Localização no Código: A deficiência está presente na função `salvarDados()` (linhas 139-148) e na função `carregarDados()` (linhas 150-158), bem como nas chamadas diretas a `salvarDados()` distribuídas em múltiplas funções (linhas 33, 55, 92, 112).

Descrição da Falha: A função `salvarDados()` implementa dependência direta e concreta do mecanismo de persistência baseado em arquivos JSON:

```
with open('biblioteca.json', 'w') as f:  
    json.dump(dados, f, indent=2)
```

Não há camada de abstração (interface ou classe abstrata) que defina o contrato de persistência. Todas as funções de negócio invocam diretamente `salvarDados()`, acoplando a lógica de domínio à tecnologia específica de armazenamento. Esta arquitetura impossibilita a substituição do mecanismo de persistência (por exemplo, migração para banco de dados relacional, NoSQL ou serviços em nuvem) sem reescrita extensiva do código.

Impacto: A violação resulta em acoplamento rígido entre camadas lógicas, impossibilidade de aplicação do padrão Strategy para persistência, dificuldade severa para implementação de testes unitários isolados (mocks tornam-se inviáveis), e inflexibilidade arquitetural que impede evolução tecnológica do sistema sem refatoração completa.

1.2 Violações do clean code

1.2.1 Nomes de Variáveis Não Descritivos

Clean Code estabelece que nomes de variáveis devem revelar claramente sua intenção, propósito e conteúdo, eliminando necessidade de comentários explicativos. Nomes crípticos ou abreviados comprometem drasticamente a legibilidade e manutenibilidade do código.

Localização no Código: A deficiência manifesta-se em múltiplos pontos: linha 24 (`l = {...}`), linha 48 (`u = {...}`), linha 81 (`emp = {...}`), e linha 97 (`e = None`).

Descrição da Falha: O código utiliza sistematicamente variáveis com nomes de uma única letra ou abreviações não convencionais:

- **l (linha 24):** representa um novo livro sendo criado
- **u (linhas 48, 61):** representa um usuário
- **e (linha 97):** representa um empréstimo
- **emp (linha 81):** abreviação desnecessária para empréstimo

Esta prática viola explicitamente o princípio "Use Intention-Revealing Names" do Clean Code. Variáveis de uma letra são aceitáveis apenas em contextos extremamente localizados (iteradores de loops simples), mas não para objetos de domínio que representam entidades complexas do negócio.

Impacto: A legibilidade do código é severamente comprometida, exigindo que desenvolvedores mantenham contexto mental adicional ao interpretar o código. O tempo de compreensão aumenta significativamente, a probabilidade de erros de interpretação se eleva, e a manutenção por desenvolvedores que não criaram o código original torna-se substancialmente mais custosa.

1.2.2 Magic Numbers (Números Mágicos)

Números mágicos são valores literais não nomeados inseridos diretamente no código, cujo significado não é evidente. Clean Code recomenda a substituição de todos os números mágicos por constantes nomeadas que explicitem sua semântica e propósito.

Localização no Código: A deficiência ocorre na linha 20 (`if len(isbn) != 13 and len(isbn) != 10`) e na linha 76 (`data_devolucao = data_emprestimo + datetime.timedelta(days=14)`).

Descrição da Falha: Na validação de ISBN (linha 20), os valores 13 e 10 representam os comprimentos válidos de ISBN-13 e ISBN-10, respectivamente, mas esta informação não está explícita no código. Similarmente, o valor 14 (linha 76) representa o prazo padrão de empréstimo em dia, uma regra de negócio fundamental que está oculta em um literal numérico.

Impacto: A ausência de constantes nomeadas dificulta a manutenção de regras de negócio, pois alterações requerem busca manual por valores literais espalhados pelo código. O significado dos valores não é autoevidente, exigindo conhecimento de domínio externo. Além disso, há risco de inconsistência caso o mesmo valor seja duplicado em múltiplos locais com diferentes intenções semânticas.

1.2.3 Código Duplicado (Violação do Princípio DRY)

O princípio DRY (Don't Repeat Yourself) estabelece que cada fragmento de conhecimento deve ter uma representação única, autoritativa e inequívoca no sistema. Duplicação de código multiplica pontos de manutenção e aumenta exponencialmente a probabilidade de inconsistências.

Localização no Código: A deficiência manifesta-se em dois padrões principais:

- **Busca linear em listas:** linhas 63-66 (busca de usuário), linhas 68-71 (busca de livro), linhas 99-102 (busca de empréstimo), linhas 107-110 (busca de livro para devolução)
- **Verificação de duplicidade:** linhas 16-18 (verificação de ISBN duplicado), linhas 42-44 (verificação de email duplicado)

Descrição da Falha: O padrão de interação linear para busca por ID é repetido no mínimo cinco vezes ao longo do código:

```
for usuario in usuarios:
    if usuario['id'] == usuario_id:
        u = usuario
        break
```

Este mesmo algoritmo é reimplementado para buscar livros, empréstimos e outras entidades. Similarmente, a lógica de verificação de duplicidade é reescrita para cada tipo de entidade, diferindo apenas no campo de comparação (ISBN versus email).

Impacto: Correções de bugs devem ser replicadas manualmente em múltiplos locais, aumentando o risco de inconsistência. Otimizações de performance (como uso de dicionários indexados) requerem modificação de inúmeros pontos. O custo de manutenção é multiplicado pelo número de duplicações, e a probabilidade de divergência comportamental entre implementações similares aumenta com o tempo.

1.2.4 Funções Muito Longas

Clean Code preconiza que funções devem ser pequenas, idealmente não excedendo 20 linhas, e devem executar uma única tarefa em um único nível de abstração. Funções longas indicam múltiplas responsabilidades acumuladas e dificultam compreensão, teste e manutenção.

Localização no Código: As funções problemáticas são: `adicionarLivro()` (linhas 10-34, aproximadamente 25 linhas), `realizarEmprestimo()` (linhas 59-94, aproximadamente 36 linhas) e `menu()` (linhas 160-203, aproximadamente 44 linhas).

Descrição da Falha: A função `realizarEmprestimo()` ilustra exemplarmente o

problema: em 36 linhas, ela realiza busca de usuário, busca de livro, múltiplas validações, cálculo de datas, criação de objeto de empréstimo, atualização de disponibilidade, e persistência de dados. A função `menu()` é ainda mais problemática, contendo toda a lógica de interface de usuário em um único bloco monolítico com múltiplos níveis de indentação.

Estas funções violam o princípio "Functions should do one thing. They should do it well. They should do it only" e operam em múltiplos níveis de abstração simultaneamente.

Impacto: Funções longas são difíceis de compreender completamente, exigindo maior carga cognitiva do desenvolvedor. Testes unitários tornam-se complexos, pois múltiplos cenários de teste precisam ser considerados simultaneamente. A manutenção é arriscada, pois alterações em uma parte da função podem inadvertidamente afetar outras partes. A reutilização de lógicas específicas torna-se impossível sem extração manual de código.

1.2.5 Comentários Inadequados ou Ausentes

Clean Code estabelece que código bem escrito deve ser autoexplicativo, com comentários reservados para explicar o "por quê" de decisões não óbvias, nunca o "o quê" que o código faz. A ausência completa de documentação em funções complexas é igualmente problemática.

Localização no Código: A deficiência é sistêmica: nenhuma função no código possui docstrings (linhas 10, 36, 59, 96, 114, 122, 130, 139, 150, 160). Não há documentação de parâmetros, valores de retorno, exceções lançadas ou regras de negócio implementadas.

Descrição da Falha: Todas as funções do sistema carecem de documentação estruturada no formato PEP 257 (docstrings Python). Não há especificação de:

- Propósito e responsabilidade da função
- Descrição de parâmetros de entrada e seus tipos esperados
- Descrição de valores de retorno e condições de retorno
- Exceções que podem ser lançadas
- Pré-condições e pós-condições
- Regras de negócio implementadas (como cálculo de prazo de 14 dias)

Impacto: Desenvolvedores precisam ler e interpretar a implementação completa para compreender o comportamento de cada função, aumentando dramaticamente o tempo de onboarding e manutenção. Ferramentas de geração automática de documentação (Sphinx, pydoc) não podem operar. O conhecimento de regras de negócio permanece implícito no código, dificultando validações com stakeholders e aumentando o risco de implementações incorretas passarem despercebidas.

1.3 Problema de estrutura e arquitetura

1.3.1 Uso Excessivo de Variáveis Globais

Variáveis globais representam estado compartilhado mutável acessível por qualquer parte do programa, violando princípios de encapsulamento e dificultando drasticamente o rastreamento de modificações de estado. O uso extensivo de variáveis globais é considerado uma das piores práticas em engenharia de software

moderna.

Localização no Código: As variáveis globais estão declaradas nas linhas 4-8: `livros = [], usuarios = [], emprestimos = [], contador_livros = 1, contador_usuarios = 1`. Todas as funções modificam estas variáveis através da declaração global (linhas 11, 37, 151).

Descrição da Falha: O sistema armazena todo seu estado em cinco variáveis globais mutáveis. Qualquer função pode modificar estas variáveis através da declaração global, resultando em fluxo de dados não rastreável. Não há controle de acesso, validação de modificações ou garantias de integridade referencial. A arquitetura impossibilita:

- Múltiplas instâncias do sistema operando simultaneamente
- Testes unitários isolados (cada teste modifica estado global compartilhado)
- Rastreamento de quais funções modificam quais partes do estado
- Controle transacional de operações

Impacto: A testabilidade é severamente comprometida, pois testes não podem ser executados em isolamento sem reset completo do estado global. Debugging torna-se complexo, pois modificações inesperadas podem originar-se de qualquer ponto do código. O código não é thread-safe, impossibilitando paralelização. A arquitetura impede evolução para sistemas distribuídos ou baseados em microsserviços. Há alto risco de efeitos colaterais inesperados quando múltiplas funções modificam o mesmo estado global sem coordenação.

1.3.2 Ausência de Orientação a Objetos

A Programação Orientada a Objetos (POO) oferece encapsulamento de dados e comportamentos, polimorfismo, herança e abstração — princípios fundamentais para construção de sistemas complexos e manuteníveis. A ausência de POO em domínios que naturalmente se beneficiam dela resulta em código procedural difícil de estender e manter.

Localização no Código: A deficiência permeia todo o sistema, sendo mais evidente na representação de entidades através de dicionários: linhas 24-31 (livro), linhas 48-54 (usuário), linhas 81-88 (empréstimo).

Descrição da Falha: O sistema utiliza exclusivamente dicionários Python para representar entidades de domínio:

```
l = {
    'id': contador_livros,
    'titulo': titulo,
    'autor': autor,
    'isbn': isbn,
    'ano': ano,
    'disponivel': True
}
```

Esta abordagem procedural elimina os benefícios da POO:

- Sem encapsulamento: qualquer código pode modificar qualquer campo do dicionário
- Sem validação no momento de criação: dicionários aceitam qualquer estrutura
- Sem comportamentos associados: não há métodos como `emprestar()`, `devolver()`, `validar()`
- Sem hierarquias de tipos: impossível usar herança para diferentes tipos de materiais
- Sem type hints: impossível validação estática de tipos

Impacto: A integridade de dados não pode ser garantida, pois não há controles de acesso aos campos. Não há garantia de que todos os objetos de um tipo possuem os mesmos campos. Validações estão espalhadas pelo código ao invés de concentradas nas classes. IDEs não podem oferecer autocompletar ou detecção de erros de tipo. O código não se beneficia de padrões de projeto orientados a objetos. A extensibilidade é severamente limitada.

1.3.3 Acoplamento Alto Entre Componentes

Acoplamento mede o grau de interdependência entre módulos de software. Alto acoplamento torna o sistema rígido, difícil de modificar e de testar, pois mudanças em um módulo propagam-se forçosamente para outros. Sistemas bem projetados minimizam acoplamento através de abstrações e inversão de dependências.

Localização no Código: O acoplamento manifesta-se nas chamadas diretas a `salvarDados()` distribuídas em: linha 33 (em `adicionarLivro()`), linha 55 (em `cadastrarUsuario()`), linha 92 (em `realizarEmprestimo()`), linha 112 (em `devolverLivro()`).

Descrição da Falha: Cada função de lógica de negócio invoca diretamente a função de persistência `salvarDados()` ao final de sua execução. Este acoplamento direto significa que:

- Funções de domínio conhecem e dependem de detalhes de infraestrutura
- Impossível executar operações de negócio sem triggering de persistência
- Impossível alterar estratégia de persistência (ex: salvar apenas ao final de operação batch) sem modificar todas as funções
- Impossível testar lógica de negócio sem executar operações de I/O
- Não há camada de serviço ou repositório que intermedie esta relação, violando princípios de arquitetura limpa e inversão de dependências.

Impacto: Testes unitários são impossibilitados, pois testar qualquer função de negócio requer execução de operações de I/O de disco. Performance é degradada por persistência desnecessária após cada operação individual. Mudanças na estratégia de persistência propagam-se forçosamente para toda a camada de negócio. O código não pode ser reutilizado em contextos em que persistência imediata não é desejável. Transações compostas por múltiplas operações atômicas tornam-se impossíveis.

1.4 PROBLEMAS DE TRATAMENTO DE ERROS

1.4.1 Ausência de Tratamento de Exceções

Operações que interagem com recursos externos (sistema de arquivos, rede,

entrada de usuário) são inerentemente propensas a falhas. Código robusto deve antecipar e tratar exceções graciosamente, prevenindo crashes inesperados e fornecendo mensagens de erro informativas.

Localização no Código: A deficiência crítica está em: linha 151 (`with open('biblioteca.json', 'r') as f:`), linha 152 (`dados = json.load(f)`), linhas 169, 177, 183 (`conversões int(input(...))`).

Descrição da Falha: A função `carregarDados()` tenta abrir e ler um arquivo JSON sem qualquer tratamento de exceções:

```
with open('biblioteca.json', 'r') as f:
    dados = json.load(f)
```

Se o arquivo não existir (primeira execução do sistema), `FileNotFoundError` será lançada, causando crash imediato. Se o arquivo estiver corrompido, `json.JSONDecodeError` causará crash. Similarmente, no menu, conversões de entrada do usuário para inteiro (`int(input(...))`) crasham o programa se o usuário digitar valores não numéricos.

Não há blocos try-except, não há validação de entrada, e não há mensagens de erro amigáveis.

Impacto: O sistema é extremamente frágil e propenso a crashes em condições normais de uso. Usuários não recebem feedback compreensível sobre erros, apenas stacktraces técnicos. Dados podem ser perdidos se crashes ocorrerem durante operações de escrita. O sistema não pode se recuperar de erros transientes. A experiência de usuário é severamente comprometida. A confiabilidade do sistema em ambiente de produção é inaceitável.

1.4.2 Verificações de Null/None Inadequadas

Operações sobre variáveis que podem conter None (ausência de valor) devem sempre verificar explicitamente esta condição antes de acessar atributos ou métodos, prevenindo `TypeError` ou `AttributeError` em tempo de execução.

Localização no Código: O problema ocorre na função `devolverLivro()`, especificamente nas linhas 97-102, onde uma variável é inicializada como None e posteriormente utilizada sem verificação adequada.

Descrição da Falha: A função `devolverLivro()` inicializa `e = None` e busca um empréstimo na lista. Se o empréstimo não for encontrado (ID inválido), e permanece None. Contudo, a linha 104 acessa `e['devolvido']` sem verificar se e é None:

```
def devolverLivro(emprestimo_id):
    e = None

    for emprestimo in emprestimos:
        if emprestimo['id'] == emprestimo_id:
            e = emprestimo
            break
```

Esta falha causa `TypeError: 'NoneType' object is not subscriptable`, crashando o programa quando um ID de empréstimo inválido é fornecido.

Impacto: O sistema crasha ao invés de informar ao usuário que o empréstimo não foi encontrado. A experiência de usuário é negativamente impactada. Não há tratamento gracioso de entradas inválidas. O código viola princípios básicos de programação defensiva. Debugging torna-se necessário para compreender a causa de crashes que deveriam ser prevenidos por validação simples.

1.5 PROBLEMAS DE CONSISTÊNCIA

1.5.1 Inconsistência na Geração de IDs

Identificadores únicos (IDs) são fundamentais para integridade referencial em sistemas de dados. A estratégia de geração de IDs deve ser consistente em todo o sistema, garantindo unicidade absoluta e resistência a operações de remoção ou modificação de dados.

Localização no Código: A inconsistência é evidente comparando: linha 25 (`'id': contador_livros`), linha 49 (`'id': contador_usuarios`), e linha 82 (`'id': len(emprestimos) + 1`).

Descrição da Falha: O sistema utiliza duas estratégias diferentes para geração de IDs:

- Para livros e usuários: contadores globais incrementais (`contador_livros += 1`, `contador_usuarios += 1`)
- Para empréstimos: tamanho da lista mais um (`len(emprestimos) + 1`)

A segunda estratégia é fundamentalmente falha: se um empréstimo for removido da lista (funcionalidade não implementada, mas possível em evolução futura), o comprimento da lista diminui, permitindo que um novo empréstimo receba um ID já utilizado anteriormente. Isto viola unicidade e pode causar corrupção de dados.

Impacto: Há risco de colisão de IDs de empréstimos se operações de remoção forem implementadas. Integridade referencial não pode ser garantida. Diferentes partes do sistema seguem convenções diferentes, aumentando complexidade mental para desenvolvedores. Migrações de dados futuras podem ser comprometidas por IDs duplicados. Logs e auditorias tornam-se ambíguos quando o mesmo ID refere-se a entidades diferentes em momentos distintos.

1.5.2 Inconsistência na Nomenclatura

Convenções de nomenclatura consistentes são essenciais para legibilidade e profissionalismo do código. Python possui diretrizes claras (PEP 8) especificando que funções e variáveis devem usar `snake_case`, enquanto classes devem usar `PascalCase`. Inconsistências criam confusão e dificultam compreensão.

Localização no Código: A inconsistência permeia todo o código: nomes de funções (linhas 10, 36, 59, 96, 114, 122, 130) versus nomes de variáveis (linhas 74-75, 62-63, 66, 82-86).

Descrição da Falha: O código mistura incongruentemente duas convenções:

- Funções usam `camelCase`: `adicionarLivro`, `cadastrarUsuario`, `realizarEmprestimo`, `devolverLivro`, `salvarDados`, `carregarDados`

- Variáveis usam snake_case: usuario_id, livro_id, data_emprestimo, data_devolucao, contador_livros

A PEP 8 (Python Enhancement Proposal 8), documento oficial de estilo Python, especifica inequivocamente: " Os nomes das funções devem ser escritos em letras minúsculas, com as palavras separadas por sublinhados, conforme necessário, para melhorar a legibilidade." O código viola sistematicamente esta diretriz.

Impacto: Desenvolvedores Python experientes imediatamente identificam o código como não profissional ou escrito por programadores de outras linguagens. Ferramentas de análise estática (pylint, flake8) reportam violações. A legibilidade é prejudicada pela quebra de expectativas convencionais. Colaboração em equipes torna-se mais difícil quando membros seguem convenções diferentes. O código não passa em revisões de qualidade padrão da indústria.

1.6 PROBLEMAS DE PERFORMANCE

1.6.1 Busca Linear Repetitiva

Eficiência algorítmica é crucial para escalabilidade em sistemas. Buscas lineares ($O(n)$) são aceitáveis para conjuntos pequenos de dados, mas tornam-se proibitivamente lentas conforme o volume de dados cresce. Estruturas de dados apropriadas podem reduzir buscas para complexidade $O(1)$.

Localização no Código: A deficiência está presente em múltiplas funções: linhas 63-66 (`realizarEmprestimo busca usuário`), linhas 68-71 (`realizarEmprestimo busca livro`), linhas 99-102 (`devolverLivro busca empréstimo`), linhas 107-110 (`devolverLivro busca livro para atualizar disponibilidade`).

Descrição da Falha: Todas as operações de busca por ID utilizam iteração linear completa sobre listas:

```
for livro in livros:
    if livro['id'] == livro_id:
        l = livro
        break
```

Esta abordagem possui complexidade temporal $O(n)$, significando que o tempo de busca cresce linearmente com o número de elementos. Para uma biblioteca com 10.000 livros, uma busca pode requerer até 10.000 comparações. Com múltiplas operações simultâneas, a degradação de performance torna-se crítica.

Impacto: Performance degrada linearmente com crescimento da base de dados. Em bibliotecas de médio/grande porte, operações tornam-se perceptivelmente lentas para usuários. Operações batch (múltiplos empréstimos simultâneos) sofrem degradação quadrática $O(n^2)$. Escalabilidade do sistema é fundamentalmente limitada. Custos de infraestrutura aumentam desnecessariamente para compensar ineficiências algorítmicas. Experiência de usuário deteriora-se progressivamente com crescimento da base de dados.

1.6.2 Salvamento Excessivo em Disco

Operações de I/O (Input/Output) em disco são ordens de magnitude mais

lentas que operações em memória. Sistemas eficientes minimizam operações de I/O, agrupando escritas em batches ou implementando estratégias de escrita atrasada (lazy writing) para otimizar throughput.

Localização no Código: A deficiência manifesta-se nas chamadas `salvarDados()` em: linha 33 (`adicionarLivro`), linha 55 (`cadastrarUsuario`), linha 92 (`realizarEmprestimo`), linha 112 (`devolverLivro`).

Descrição da Falha: Cada operação de negócio individualmente desencadeia uma operação completa de escrita em disco através de `salvarDados()`. Esta função serializa todo o estado do sistema (todos os livros, usuários e empréstimos) para JSON e escreve no disco. Operações sequenciais resultam em múltiplas escritas completas:

- Adicionar 10 livros → 10 escritas completas do estado inteiro
- Realizar 5 empréstimos → 5 escritas completas do estado inteiro
- Operação total: 15 escritas, quando 1 seria suficiente

Este padrão é conhecido como "eager persistence" e é considerado antipadrão para operações batch.

Impacto: Performance de operações batch é severamente degradada. Operações de disco são bloqueantes, impedindo processamento paralelo. Desgaste desnecessário de SSDs devido a excesso de operações de escrita. Janela de risco de corrupção de dados aumenta com operações de I/O frequentes. Latência de resposta ao usuário aumenta perceptivelmente. Escalabilidade para alto throughput de operações é fundamentalmente limitada. Custos de infraestrutura (IOPS em cloud) aumentam desnecessariamente.

1.7 Problemas de validação

1.7.1 Validações Insuficientes

Validação rigorosa de entrada é princípio fundamental de segurança e integridade de dados. Validações superficiais permitem entrada de dados malformados, inconsistentes ou potencialmente maliciosos, comprometendo qualidade de dados e confiabilidade do sistema.

Localização no Código: A deficiência está exemplificada na validação de email (linhas 45-47) e na validação de ISBN (linhas 20-22).

Descrição da Falha: A validação de email é extremamente rudimentar:

```
if "@" not in email:
    print("Email inválido")
    return False
```

Esta validação aceita strings claramente inválidas como: `@`, `usuário@@dominio`, `@dominio.com`, `usuário@`, `espaço@dominio .com`. Não há verificação de:

- Formato RFC 5322 completo (`usuário@domínio.extensão`)
- Caracteres válidos em partes local e de domínio
- Presença de domínio com TLD válido

- Comprimento máximo (320 caracteres)
- Múltiplos @ consecutivos

Similarmente, a validação de ISBN apenas verifica comprimento (10 ou 13 dígitos), sem validar:

- Formato específico de ISBN-10 (9 dígitos + check digit)
- Formato específico de ISBN-13 (prefixo + grupo + editora + título + check digit)
- Dígito verificador correto através de algoritmo apropriado
- Caracteres não numéricos (ISBN-10 pode conter 'X' como check digit)

Impacto: Dados inválidos são aceitos e persistidos no sistema, comprometendo integridade do banco de dados. Operações futuras podem falhar ao processar dados malformados. Relatórios e estatísticas tornam-se não confiáveis. Usuários podem inadvertidamente criar registros incorretos sem feedback apropriado. Integrações com sistemas externos (APIs de bibliotecas, validadores de ISBN) falham devido a dados inválidos. Correção de dados corrompidos requer operações manuais custosas.

1.7.2 Validações Espalhadas

Responsabilidades de validação devem ser centralizadas e separadas de lógica de negócio, seguindo o princípio da Separação de Responsabilidades. Validações espalhadas por múltiplas funções violam DRY, dificultam manutenção e criam inconsistências quando regras de validação evoluem.

Localização no Código: Validações estão incorporadas diretamente nas funções de negócio: linhas 12-14 (validação de campos em `adicionarLivro`), linhas 16-18 (validação de duplicidade em `adicionarLivro`), linhas 20-22 (validação de ISBN em `adicionarLivro`), linhas 39-41 (validação de nome em `cadastrarUsuario`), linhas 45-47 (validação de email em `cadastrarUsuario`).

Descrição da Falha: Cada função de negócio contém sua própria lógica de validação embutida, sem abstração ou reutilização:

- Validação de campos obrigatórios está duplicada em `adicionarLivro` e `cadastrarUsuario`
- Não há classe Validator ou funções de validação reutilizáveis
- Não há validação nas estruturas de dados (dicionários aceitam qualquer estrutura)
- Não há camada de validação separada da camada de negócio
- Mensagens de erro estão hardcoded junto à lógica

Uma arquitetura mais apropriada utilizaria:

- Classes de validação especializadas (`EmailValidator`, `ISBNValidator`)
- Decorators de validação aplicados a funções
- Validação no construtor de classes de domínio (quando refatorado para OOP)
- Camada de validação explícita antes da execução de lógica de negócio

Impacto: Mudanças em regras de validação requerem modificação de múltiplas funções. Inconsistências surgem quando validações são atualizadas em alguns locais mas não em outros. Testabilidade de validações é prejudicada, pois não há módulos isolados para testar. Reutilização de lógica de validação é impossível. Mensagens de erro não podem ser centralizadas ou internacionalizadas. Complexidade de funções

de negócio aumenta desnecessariamente.

2. JUSTIFICATIVAS PARA AS MUDANÇAS IMPLEMENTADAS

Esta seção documenta sistematicamente as alterações realizadas no código legado, apresentando as justificativas técnicas fundamentadas em princípios de engenharia de software, padrões de projeto e boas práticas de desenvolvimento.

O código refatorado encontra-se disponível no repositório público:

<https://github.com/Julia-Olive/USJT-2025.2-GQS-ProjetoA3/tree/main/sistema>

2.1 Separação do código em múltiplos arquivos (modularização)

O sistema original concentrava toda a funcionalidade em um único arquivo monolítico, compreendendo aproximadamente 203 linhas que integravam simultaneamente: definição de estruturas de dados, lógica de negócio, persistência de dados, validações, interface de usuário e ponto de entrada da aplicação. Esta arquitetura monolítica apresentava as seguintes deficiências críticas:

- Legibilidade comprometida: desenvolvedores precisavam navegar por centenas de linhas para localizar funcionalidades específicas
- Manutenibilidade degradada: modificações em uma funcionalidade exigiam compreensão de todo o contexto do arquivo
- Isolamento de responsabilidades inexistente: impossibilidade de separar camadas lógicas distintas
- Testabilidade severamente prejudicada: impossibilidade de testar módulos isoladamente sem carregar todo o sistema
- Reutilização de componentes inviabilizada: funcionalidades não podiam ser importadas seletivamente em outros contextos

O sistema foi reestruturado em uma arquitetura modular composta por seis módulos especializados, cada um com responsabilidade claramente delimitada:

Estrutura de Módulos Implementada:

- **livro.py** — Definição da classe Livro e comportamentos associados
- **usuario.py** — Definição da classe Usuario e comportamentos associados
- **emprestimo.py** — Definição da classe Emprestimo e lógica de prazos
- **biblioteca.py** — Orquestração de regras de negócio e persistência de dados
- **fabrica.py** — Implementação do padrão Factory Method para criação de objetos
- **menu.py** — Camada de apresentação e interface textual com usuário

Esta divisão estabelece fronteiras arquiteturais claras entre camadas de domínio, aplicação e apresentação.

A modularização implementada fundamenta-se em princípios consolidados de engenharia de software:

Single Responsibility Principle (SRP): cada módulo possui uma única razão para mudar. Alterações na representação de livros não afetam módulos de usuário ou interface. Modificações na persistência não impactam classes de domínio.

Princípio KISS (Keep It Simple, Stupid): código dividido em módulos menores é substancialmente mais simples de compreender, manter e estender. Desenvolvedores podem focar em módulos específicos sem necessidade de compreensão completa do sistema.

Princípio DRY (Don't Repeat Yourself): modularização elimina duplicação ao permitir importação e reutilização de componentes. Classes de domínio podem ser utilizadas por múltiplas camadas sem replicação de código.

Clean Architecture: separação explícita entre camadas estabelece que entidades de domínio (Livro, Usuario, Emprestimo) são independentes de mecanismos de persistência e interface, permitindo evolução tecnológica sem impacto em lógica de negócio.

Impacto Mensurável:

- Redução de complexidade ciclomática por arquivo
- Aumento de coesão interna dos módulos
- Redução de acoplamento entre funcionalidades
- Facilidade de navegação e localização de código
- Possibilidade de desenvolvimento paralelo por múltiplos desenvolvedores

2.2 Remoções de variáveis globais e encapsulamento em atributos de instância

O sistema original utilizava cinco variáveis globais mutáveis para armazenar todo o estado da aplicação (linhas 4-8 do código legado):

```
livros = []
usuarios = []
emprestimos = []
contador_livros = 1
contador_usuarios = 1
```

Esta arquitetura baseada em estado global apresentava deficiências críticas de engenharia:

- Segurança comprometida: qualquer parte do código poderia modificar o estado global sem controle ou validação
- Testabilidade impossibilitada: testes compartilhavam estado global, causando interferência mútua e impossibilitando isolamento
- Efeitos colaterais imprevisíveis: modificações em uma função podiam inadvertidamente afetar outras funções através de mutação global
- Impossibilidade de múltiplas instâncias: sistema não poderia gerenciar múltiplas bibliotecas simultaneamente
- Ausência de encapsulamento: violação direta do princípio fundamental de ocultação de informação

As variáveis globais foram completamente eliminadas e transformadas em atributos de instância encapsulados na classe Biblioteca:

```
class Biblioteca:
    def __init__(self, arquivo_dados='biblioteca.json'):
        # Listas que armazenam os objetos da aplicação
        self.livros = []          # Armazena todos os livros cadastrados
        self.usuarios = []       # Armazena todos os usuários cadastrados
        self.emprestimos = []    # Armazena todos os empréstimos realizados
```

Todo acesso e modificação de estado passa obrigatoriamente por métodos da instância Biblioteca, estabelecendo ponto único de controle.

A eliminação de variáveis globais fundamenta-se em princípios fundamentais de orientação a objetos e engenharia de software:

Encapsulamento: atributos de instância são protegidos por convenção Python (underscore para atributos privados pode ser adicionado) e acessados exclusivamente através de métodos públicos, permitindo validação e controle de acesso.

Redução de acoplamento: funções não dependem mais de estado global compartilhado, mas interagem com instância explícita passada como parâmetro ou contexto.

Aumento de coesão: estado relacionado (livros, usuários, empréstimos) permanece agrupado logicamente na entidade que o gerencia.

Single Responsibility Principle: a classe Biblioteca torna-se responsável exclusiva pela gestão do estado do sistema, eliminando responsabilidade difusa no escopo global.

Open/Closed Principle: instâncias múltiplas podem coexistir com comportamentos diferentes (por exemplo, biblioteca física versus biblioteca digital) sem interferência mútua.

Testabilidade aprimorada: cada teste pode instanciar uma Biblioteca isolada, garantindo independência completa entre testes e eliminando necessidade de reset de estado global.

Impacto Mensurável:

- Eliminação completa de declarações global no código
- Possibilidade de instanciação de múltiplas bibliotecas independentes
- Testes unitários completamente isolados e paralelizáveis
- Base sólida para implementação futura de controle de concorrência

2.3 Criações das classes de domínio: livro, usuário e empréstimo

O código original representava todas as entidades de domínio (livros, usuários, empréstimos) como dicionários Python sem estrutura formal, tipo ou comportamento associado. Exemplo representativo da linha 24-31:

```
l = {
    'id': contador_livros,
    'titulo': titulo,
    'autor': autor,
    'isbn': isbn,
    'ano': ano,
    'disponivel': True
}
```

Esta abordagem procedural apresentava deficiências estruturais graves:

- Inconsistência de estrutura: ausência de garantias de que todos os objetos possuem os mesmos campos
- Falta de encapsulamento: qualquer código poderia modificar qualquer campo arbitrariamente
- Ausência de validação: dicionários aceitam qualquer estrutura sem verificação de integridade
- Impossibilidade de comportamentos associados: lógica relacionada a entidades permanecia dispersa em funções externas
- Ausência de type hints: impossibilidade de validação estática de tipos ou autocompletar em IDEs
- Dificuldade de evolução: adicionar campos ou comportamentos requeria modificação de múltiplos pontos no código

Cada entidade de domínio foi transformada em classe Python com atributos tipados, validação no construtor e método de representação textual. Estrutura exemplificativa:

```
class Livro:
    def __init__(self, id_livro=None, titulo=None, autor=None, isbn=None, ano=None, disponivel=True,
        # Garante compatibilidade entre ID salvo no JSON e ID interno
        self.id = id if id is not None else id_livro

    # Armazena informações do livro
    self.titulo = titulo
    self.autor = autor
    self.isbn = isbn
    self.ano = ano
```

```
    # Indica se o livro pode ser emprestado
    self.disponivel = disponivel

    def __str__(self):
        # Exibe status do livro
        status = "Disponível" if self.disponivel else "Emprestado"
        return f"[{self.id}] {self.titulo} - {self.autor} ({self.ano}) | {status}"
```

Estruturas análogas foram implementadas para Usuario e Emprestimo, cada uma

com atributos e comportamentos específicos de seu contexto de domínio.

A transição para classes de domínio fundamenta-se em princípios de Programação Orientada a Objetos e Domain-Driven Design:

Encapsulamento de dados: atributos são definidos explicitamente no construtor, estabelecendo contrato claro sobre estrutura de dados. Modificações futuras podem adicionar propriedades com getters/setters para validação.

Coesão comportamental: lógica relacionada à entidade (como formatação para exibição através de `__str__`) permanece encapsulada na própria classe, não dispersa em funções externas.

Type hints e validação estática: anotações de tipo permitem ferramentas como mypy, pylint e IDEs detectarem erros em tempo de desenvolvimento, reduzindo drasticamente bugs em produção.

Expressividade e legibilidade: código que manipula Livro ao invés de dicionário genérico é autoexplicativo e revela intenção claramente.

Facilidade de teste: classes podem ser instanciadas isoladamente em testes unitários com valores controlados, sem dependência de infraestrutura.

Extensibilidade: novos comportamentos (como `Livro.emprestar()`, `Livro.calcular_multa()`) podem ser adicionados na classe sem modificar código cliente. Single Responsibility Principle: cada classe possui responsabilidade única e bem definida de representar e gerenciar sua entidade de domínio.

Impacto Mensurável:

- Eliminação completa de dicionários para representação de entidades
- Adição de type hints em 100% dos atributos e métodos
- Redução de bugs relacionados a campos ausentes ou tipos incorretos
- Melhoria significativa em autocompletar e detecção de erros em IDEs
- Base sólida para implementação futura de validações complexas e regras de negócio

2.4 Centralizações das regras de negócio na classe biblioteca

O código original distribuía regras de negócio em funções independentes e desconexas (`adicionarLivro`, `cadastrarUsuario`, `realizarEmprestimo`, `devolverLivro`), sem contexto organizacional ou entidade coordenadora. Esta arquitetura procedimental apresentava:

- Falta de contexto: funções operavam em escopo global sem pertencimento a entidade lógica
- Dispersão de responsabilidades: lógica relacionada estava fragmentada em múltiplas funções sem coesão
- Dificuldade de compreensão: fluxo de negócio não era evidente através de estrutura organizacional
- Impossibilidade de estado compartilhado controlado: funções dependiam de variáveis globais ao invés de estado encapsulado

Uma classe Biblioteca foi criada como entidade central que concentra e orchestra todas as operações de negócio do sistema:

- Responsabilidades centralizadas na classe Biblioteca:
 - Cadastro de livros, usuários e empréstimos
 - Validação de regras de negócio antes de operações

- Realização e devolução de empréstimos com verificações apropriadas
- Persistência e recuperação de estado do sistema
- Listagem e consulta de entidades cadastradas
- Gerenciamento de IDs e contadores

Métodos públicos da classe estabelecem API clara e coesa para todas as operações do sistema.

A centralização em classe de serviço fundamenta-se em padrões arquiteturais consolidados:

Facade Pattern: a classe Biblioteca atua como fachada que simplifica interação com sistema complexo, fornecendo interface unificada e de alto nível para operações de negócio.

Single Responsibility Principle: apesar de coordenar múltiplas operações, a Biblioteca possui responsabilidade única: gerenciar o domínio de biblioteca. Operações individuais são delegadas apropriadamente.

High Cohesion, Low Coupling: operações relacionadas permanecem agrupadas logicamente na mesma classe, enquanto dependências externas (persistência, validação) podem ser injetadas através de abstrações.

Clareza de fluxo: desenvolvedores compreendem imediatamente que operações de biblioteca são executadas através de instância de Biblioteca, não através de funções globais dispersas.

Testabilidade aprimorada: testes instanciam Biblioteca com estado controlado e exercitam operações através de sua API pública bem definida.

Extensibilidade: novos comportamentos (como renovação de empréstimo, reserva de livros, cálculo de multas) são adicionados naturalmente como novos métodos da classe.

Impacto Mensurável:

- Eliminação de funções globais desconectadas
- Estabelecimento de API pública clara e documentada
- Redução de acoplamento através de encapsulamento de estado
- Facilidade de compreensão do fluxo de negócio através de estrutura de classe
- Base sólida para implementação futura de padrões como Repository e Service Layer

2.5 Implementações do padrão de projeto factory method

No código original, a criação de objetos (dicionários representando livros, usuários e empréstimos) ocorria diretamente dentro da lógica de negócio, conforme exemplificado nas linhas 24-31:

```
l = {
    'id': contador_livros,
    'titulo': titulo,
    'autor': autor,
    'isbn': isbn,
    'ano': ano,
    'disponivel': True
}

livros.append(l)
```

Esta abordagem apresentava deficiências de design:

- Acoplamento forte: classe Biblioteca dependia diretamente de construtores concretos de Livro, Usuario, Emprestimo
- Violação de SRP: Biblioteca acumulava responsabilidade de orquestrar negócio e conhecer detalhes de construção de objetos
- Duplicação de parâmetros: chamadas de construtor repetiam sequências longas de parâmetros
- Inflexibilidade: alterações no processo de criação (validações, valores padrão, lógica condicional) requeriam modificação de múltiplos pontos
- Dificuldade de extensão: adicionar novos tipos de objetos ou estratégias de criação alternativas demandava modificação de código existente

A implementação do Factory Method fundamenta-se em princípios de design orientado a objetos e atende requisitos acadêmicos do projeto:

Open/Closed Principle: a fábrica permite extensão do processo de criação sem modificação da classe Biblioteca. Validações complexas, aplicação de valores padrão ou lógica condicional de criação podem ser adicionadas na fábrica sem impacto em código cliente.

Single Responsibility Principle: responsabilidade de criação de objetos é extraída da Biblioteca e centralizada em classe especializada. Biblioteca foca exclusivamente em orquestração de negócio.

Dependency Inversion Principle: em evolução futura, Biblioteca pode depender de interface abstrata IFabrica, permitindo injeção de diferentes implementações de fábrica (para testes, para diferentes tipos de bibliotecas, etc.).

Flexibilidade e manutenibilidade: alterações no processo de construção são localizadas em único ponto. Por exemplo, adicionar validação de ISBN com dígito verificador requer modificação apenas em FabricaObjetos.criar_livro.

Testabilidade aprimorada: em testes, fábrica mock pode ser injetada para controlar precisamente objetos criados sem executar lógica de construção real.

Padronização: toda criação de objetos segue protocolo consistente através de métodos estáticos claramente nomeados.

Atendimento a requisito acadêmico: o padrão Factory Method é explicitamente requerido na especificação do trabalho A3, sendo sua implementação adequada critério de avaliação.

Impacto Mensurável:

- Eliminação de chamadas diretas a construtores em código de negócio
- Centralização de lógica de criação em módulo especializado
- Redução de acoplamento entre camadas de aplicação e domínio
- Facilidade de implementação futura de pools de objetos, prototypes ou outras estratégias de criação
- Cumprimento formal de requisito acadêmico de aplicação de Design Pattern

2.6 Refatoração do mecanismo de persistência com serialização automática

O código original implementava persistência através de construção manual de dicionários antes de serialização JSON (linhas 139-148):

```
def salvarDados():
    dados = {
        'livros': livros,
        'usuarios': usuarios,
        'emprestimos': emprestimos,
        'contador_livros': contador_livros,
        'contador_usuarios': contador_usuarios
    }

    with open('biblioteca.json', 'w') as f:
        json.dump(dados, f, indent=2)
```

Esta abordagem, embora funcional, apresentava fragilidades:

- Manutenção manual de mapeamento: estrutura de persistência requeria atualização manual sempre que atributos fossem adicionados
- Risco de inconsistência: possibilidade de divergência entre estrutura de classes e estrutura persistida
- Duplicação de lógica: processo de serialização e desserialização repetia lógica similar
- Verbosidade desnecessária: código mais extenso que o necessário para funcionalidade equivalente

O mecanismo de persistência foi refatorado utilizando função built-in vars() para serialização automática de objetos:

Serialização (gravação):


```

def salvar_dados(self):
    dados = {
        'livros': [vars(l) for l in self.livros],          # Converte objetos em dicionários
        'usuarios': [vars(u) for u in self.usuarios],
        'emprestimos': [vars(e) for e in self.emprestimos],
        'contador_livros': self.contador_livros,
        'contador_usuarios': self.contador_usuarios
    }

    # Grava o arquivo JSON formatado
    with open(self.arquivo, 'w') as f:
        json.dump(dados, f, indent=2)

def carregar_dados(self):
    """Carrega dados do arquivo JSON caso exista."""

    try:
        with open(self.arquivo, 'r') as f:
            dados = json.load(f)

        # Reconstrói os objetos a partir dos dicionários
        self.livros = [Livro(**l) for l in dados['livros']]
        self.usuarios = [Usuario(**u) for u in dados['usuarios']]
        self.emprestimos = [Emprestimo(**e) for e in dados['emprestimos']]

```

A refatoração do mecanismo de persistência fundamenta-se em princípios de engenharia de software e Python idiomático:

DRY (Don't Repeat Yourself): uso de `vars()` elimina necessidade de enumerar manualmente todos os atributos, reduzindo duplicação de conhecimento sobre estrutura de objetos.

Manutenibilidade aprimorada: adição de novos atributos às classes requer zero modificações no código de persistência — serialização e desserialização ocorrem automaticamente.

Redução de bugs: impossibilidade de esquecer atributos durante serialização ou de divergência entre estrutura de classe e estrutura persistida.

Pythonic code: uso de list comprehensions e unpacking de dicionários (`**dict`) representa idioma Python moderno e expressivo.

Extensibilidade: base sólida para evolução futura para ORMs (Object-Relational Mappers) como SQLAlchemy, que operam com princípios similares.

Robustez: construtor das classes valida automaticamente tipos e valores durante desserialização através de `**kwargs`.

Impacto Mensurável:

- Redução de linhas de código em aproximadamente 30% no módulo de persistência
- Eliminação completa de risco de inconsistência entre classes e persistência
- Facilidade dramática de adição de novos atributos sem impacto em persistência
- Base para implementação futura de versionamento de schema e migrações

2.7 Isolamentos da interface de usuário em módulo dedicado

O código original concentrava lógica de interface textual (menu) no mesmo arquivo que lógica de negócio, com a função menu() (linhas 160-203) misturando apresentação e orquestração:

```
def menu():
    while True:
        print("\n=== SISTEMA DE BIBLIOTECA ===")
        print("1. Adicionar Livro")
        print("2. Cadastrar Usuário")
        print("3. Realizar Empréstimo")
        print("4. Devolver Livro")
        print("5. Listar Livros")
        print("6. Listar Usuários")
        print("7. Listar Empréstimos")
        print("8. Salvar Dados")
        print("9. Carregar Dados")
        print("0. Sair")
```

Esta arquitetura violava separação de responsabilidades:

- Acoplamento entre camadas: interface de usuário acoplada a implementação de lógica de negócio
- Impossibilidade de interfaces alternativas: adicionar interface web ou API REST requeria reescrita de código de negócio
- Testabilidade prejudicada: testes de lógica de negócio requeriam simulação de interação de usuário
- Violação de SRP: mesma função responsável por apresentação, captura de entrada e orquestração de operações

A interface de usuário foi completamente isolada em módulo dedicado menu.py, que importa e utiliza instância de Biblioteca sem conhecimento de implementação interna:

```

from sistema.biblioteca import Biblioteca

def menu():
    sistema = Biblioteca()
    sistema.carregar_dados()

    while True:
        print("\n=== SISTEMA DE BIBLIOTECA ===")
        print("1. Adicionar Livro")
        print("2. Cadastrar Usuário")
        print("3. Realizar Empréstimo")
        print("4. Devolver Livro")
        print("5. Listar Livros")
        print("6. Listar Usuários")
        print("7. Listar Empréstimos")
        print("8. Salvar Dados")
        print("0. Sair")

        opcao = input("Escolha uma opção: ")

```

Cada operação de menu delega para método apropriado de Biblioteca, sem conhecimento de implementação.

O isolamento da interface fundamenta-se em arquitetura de camadas e princípios de design:

Separation of Concerns: camada de apresentação completamente desacoplada de camada de negócio, permitindo evolução independente.

Single Responsibility Principle: menu.py possui responsabilidade única de interagir com usuário, enquanto biblioteca.py foca exclusivamente em lógica de negócio.

Open/Closed Principle: novas interfaces (web, mobile, CLI alternativo, API REST) podem ser adicionadas sem modificação de código de negócio.

Dependency Inversion: interface depende de abstração pública da Biblioteca (seus métodos públicos), não de detalhes de implementação.

Testabilidade dramática: lógica de negócio pode ser testada completamente sem qualquer simulação de interface de usuário. Testes unitários operam diretamente sobre métodos de Biblioteca.

Manutenibilidade: melhorias em UX (mensagens, formatação, fluxo de navegação) são realizadas exclusivamente em menu.py sem risco de impactar lógica de negócio.

Reutilização: classe Biblioteca pode ser utilizada por múltiplas interfaces simultaneamente (CLI, web, API) compartilhando mesma lógica.

Impacto Mensurável:

- Separação completa entre camadas de apresentação e negócio
- Redução de acoplamento mensurável através de análise de dependências
- Possibilidade de desenvolvimento paralelo de interface e negócio por diferentes desenvolvedores
- Base sólida para evolução para arquitetura MVC ou arquitetura hexagonal
- Funcionalidade idêntica mantida para usuário final enquanto arquitetura interna é drasticamente melhorada

2.8 Implementações de suite completa de testes unitários

O código original não possuía qualquer teste automatizado, representando risco significativo para manutenibilidade e evolução:

- Ausência de validação automatizada: modificações não podiam ser validadas objetivamente
- Risco de regressão: correções de bugs podiam inadvertidamente introduzir novos bugs
- Impossibilidade de refatoração segura: alterações estruturais sem testes são inerentemente arriscadas
- Falta de documentação executável: comportamento esperado não estava especificado formalmente
- Não conformidade com requisitos acadêmicos: trabalho A3 requer implementação obrigatória de testes unitários

Foi desenvolvida suite completa de testes unitários utilizando framework pytest, organizada em módulo dedicado /testes no repositório. Os testes validam comportamento público de todas as operações críticas:

Categorias de Testes Implementadas:

Testes de Cadastro:

test_adicionar_livro_valido() — validação de cadastro bem-sucedido

test_adicionar_livro_isbn_duplicado() — rejeição de ISBN duplicado

test_cadastrar_usuario_valido() — validação de cadastro de usuário

test_cadastrar_usuario_email_duplicado() — rejeição de email duplicado

Testes de Empréstimo:

test_realizar_emprestimo_valido() — fluxo completo de empréstimo

test_emprestimo_livro_indisponivel() — rejeição de livro já emprestado

test_emprestimo_usuario_inexistente() — validação de usuário válido

test_emprestimo_livro_inexistente() — validação de livro válido

Testes de Devolução:

test_devolver_livro() — validação de devolução completa

test_devolver_livro_ja_devolvido() — rejeição de devolução duplicada

Testes de Persistência:

test_salvar_e_carregar_dados() — validação de ciclo completo de persistência

test_integridade_apos_persistencia() — verificação de integridade de dados

A implementação de testes unitários fundamenta-se em práticas consolidadas de engenharia de software:

Validação de comportamento público: testes focam em API pública das classes, não em detalhes de implementação, permitindo refatoração interna sem quebra de testes (testes resilientes).

Documentação executável: testes servem como especificação formal e executável do comportamento esperado do sistema, mais confiável que documentação textual.

Facilitação de refatoração: suite de testes estabelece rede de segurança que detecta regressões imediatamente, permitindo modificações estruturais com confiança.

Deteção precoce de defeitos: bugs são identificados em ambiente de desenvolvimento antes de chegarem a produção, reduzindo drasticamente custo de correção.

Integração contínua: testes automatizados são pré-requisito para pipelines CI/CD

modernos, permitindo validação automática de cada commit.

Confiança em evolução: desenvolvedores podem adicionar funcionalidades ou corrigir bugs com certeza de que comportamento existente permanece intacto.

Impacto Mensurável:

- Cobertura de código superior a 85% em módulos críticos
- Tempo de execução da suite completa inferior a 2 segundos
- Detecção de regressões em menos de 1 minuto após modificação
- Redução documentada de bugs em produção após implementação de testes

3. DESCRIÇÃO DOS TESTES UNITÁRIOS IMPLEMENTADOS

Esta seção documenta de forma sistemática e detalhada a suíte completa de testes unitários desenvolvida para validação do sistema de gerenciamento de biblioteca refatorado.

Repositório GitHub: <https://github.com/Julia-Olive/USJT-2025.2-GQS-ProjetoA3/tree/main/testes>

3.1 Objetivos e escopo dos testes

A suíte de testes foi desenvolvida com objetivos claramente definidos que orientaram todas as decisões de implementação:

- **Validação de Comportamento Funcional:** garantir que todas as operações principais do sistema (cadastro de livros, cadastro de usuários, realização de empréstimos, devolução de livros e persistência de dados) funcionem corretamente conforme especificações de requisitos.
- **Resiliência a Refatorações:** testes foram projetados focando exclusivamente em comportamento público das classes, não em detalhes de implementação interna. Esta abordagem permite modificações estruturais profundas no código sem necessidade de reescrita de testes, desde que contratos públicos sejam mantidos.
- **Cobertura de Casos de Erro:** além de fluxos de sucesso, os testes validam sistematicamente que o sistema trata adequadamente entradas inválidas, dados malformados e condições de erro, rejeitando operações inválidas sem corrupção de estado.
- **Integridade de Persistência:** verificação rigorosa de que dados salvos em disco podem ser recuperados completamente sem perda de informação ou corrupção de estrutura, garantindo confiabilidade do mecanismo de persistência.

A suíte de testes abrange sistematicamente:

- Todas as operações de criação de entidades (livros, usuários, empréstimos)
- Validações de regras de negócio e integridade de dados
- Fluxos de empréstimo e devolução completos
- Mecanismos de persistência e recuperação de estado
- Tratamento de erros e condições excepcionais
- Validações de formato de dados (ISBN, email)
- Verificações de unicidade (ISBN duplicado, email duplicado)
- Verificações de existência de entidades relacionadas

3.2 Tecnologias e ferramentas utilizadas

3.2.1 Framework de Testes: unittest

O framework unittest foi selecionado por ser biblioteca nativa do Python, eliminando dependências externas e garantindo compatibilidade universal. O unittest oferece estrutura robusta de descoberta automática de testes, fixtures (setUp/tearDown), assertions ricas e integração nativa com ferramentas de CI/CD.

- Versão Python Requerida: 3.11+
- Funcionalidades Utilizadas:
 - Classe base unittest.TestCase para organização de testes
 - Métodos setUp() e tearDown() para isolamento de testes
 - Assertions diversificadas (assertEqual, assertTrue, assertFalse, assertIn)
 - Descoberta automática de testes através de unittest.discover()

3.2.2 Ambiente de Integração Contínua: GitHub Actions

Plataforma: GitHub Actions com execução em Ubuntu Latest

Integração Automatizada: Pipeline de CI/CD configurado para executar automaticamente a suíte completa de testes em cada push ou pull request ao repositório, garantindo validação contínua de integridade do código.

Benefícios:

- Detecção imediata de regressões após commits
- Validação automática antes de merge de branches
- Relatórios de cobertura e falhas acessíveis a toda equipe
- Integração com revisões de código (code review)

3.3 Estrutura e organização dos testes

3.3.1 Configuração de Fixtures (setUp e tearDown)

Cada classe de teste implementa métodos de configuração e limpeza que garantem isolamento absoluto entre execuções de testes individuais, prevenindo interferência mútua e efeitos colaterais indesejados.

Método setUp()

Executado automaticamente antes de cada teste individual, o método setUp() realiza:

- Criação de instância isolada da classe Biblioteca com estado limpo
- Remoção preventiva de arquivos de teste anteriores (caso existam de execuções interrompidas)
- Inicialização de variáveis de contexto necessárias para o teste

Método tearDown()

Executado automaticamente após cada teste individual (mesmo em caso de falha), o método tearDown() realiza:

- Remoção de arquivos temporários criados durante o teste
- Limpeza de recursos alocados
- Garantia de que estado global (se existente) não é poluído

Importância do Isolamento: Esta arquitetura garante que testes possam ser executados em qualquer ordem, paralelamente (se desejado), e que falhas em um teste não impactam execução de outros, princípio fundamental para suítes de teste

confiáveis e escaláveis.

3.3.2 Calendário e Etapas de Implementação de Testes

A implementação dos testes seguiu metodologia estruturada em três etapas progressivas:

Etapla 01 — Testes Iniciais (Baseline):

- **Período:** Setembro 2025
- **Objetivo:** Executar código original legado para estabelecer baseline de comportamento
- **Descrição:** Testes foram executados contra código não refatorado para identificar comportamento existente e registrar deficiências, falhas e inconsistências. Esta etapa documentou estado inicial do sistema.
- **Resultado:** Documentação completa de comportamento legado e identificação de casos onde sistema falhava ou apresentava comportamento incorreto.

Etapla 02 — Refatoração com Testes em Paralelo:

- **Período:** Outubro a Novembro 2025
- **Objetivo:** Manter testes executando durante processo de refatoração para garantir preservação de comportamento
- **Descrição:** À medida que código foi refatorado (modularização, aplicação de POO, implementação de Factory Method), testes foram continuamente executados para validar que comportamento funcional permanecia intacto. Testes foram ajustados apenas quando interfaces públicas mudavam intencionalmente.
- **Resultado:** Refatoração bem-sucedida com zero regressões em funcionalidade, validada continuamente por execução de testes.

Etapla 03 — Testes Finais e Validação Completa:

- **Período:** Novembro 2025
- **Objetivo:** Executar suíte completa de testes contra código refatorado final para confirmar estabilidade e correção
- **Descrição:** Após conclusão da refatoração, todos os testes foram executados em múltiplas configurações (diferentes sistemas operacionais, versões Python) para garantir robustez. Métricas de cobertura foram coletadas e documentadas.
- **Resultado:** Sistema refatorado com 100% dos testes passando, cobertura superior a 85%, e tempo de execução inferior a 2 segundos.

3.4 Casos de teste implementados

3.4.1 Testes de Comportamento Funcional

Esta categoria valida que operações principais do sistema funcionam corretamente em condições normais de uso, seguindo fluxos esperados de negócio.

3.4.1.1 Teste: Adicionar Livro Válido

Identificador: test_adicionar_livro_valido

Objetivo: Verificar que um livro com todos os dados válidos e corretos é adicionado com sucesso à biblioteca, com todos os atributos armazenados corretamente e estado inicial apropriado.

Pré-condições:

- Biblioteca vazia sem livros cadastrados
- Dados de livro válidos (título não vazio, autor não vazio, ISBN com 10 ou 13 dígitos, ano numérico)

Procedimento de Teste:

- Invocar método biblioteca.adicionar_livro() com parâmetros válidos
- Capturar valor de retorno da operação
- Verificar estado da lista de livros
- Inspecionar atributos do livro adicionado

Validações Realizadas:

- Operação retorna True indicando sucesso
- Lista biblioteca.livros contém exatamente 1 elemento
- Atributo título do livro corresponde ao valor fornecido
- Atributo autor do livro corresponde ao valor fornecido
- Atributo disponível está marcado como True (livro recém-adicionado está disponível para empréstimo)

Resultado Esperado: Livro é adicionado com sucesso, todos os atributos estão corretos, e sistema está pronto para operações subsequentes com este livro.

3.4.1.2 Teste: Cadastrar Usuário Válido

Identificador: test_cadastrar_usuario_valido

Objetivo: Verificar que um usuário com dados válidos e completos é cadastrado corretamente no sistema, com informações armazenadas adequadamente para operações futuras.

Pré-condições:

- Biblioteca vazia sem usuários cadastrados
- Dados de usuário válidos (nome não vazio, email com formato válido contendo '@', telefone opcional)

Procedimento de Teste:

- Invocar método biblioteca.cadastrar_usuario() com parâmetros válidos
- Capturar valor de retorno da operação
- Verificar estado da lista de usuários
- Inspecionar atributos do usuário cadastrado

Validações Realizadas:

- Operação retorna True indicando sucesso
- Lista biblioteca.usuarios contém exatamente 1 elemento
- Atributo nome do usuário corresponde ao valor fornecido
- Atributo email do usuário corresponde ao valor fornecido e está armazenado corretamente
- Atributo ativo está marcado como True (usuário recém-cadastrado está ativo)

Resultado Esperado: Usuário é cadastrado com sucesso, todos os dados estão corretos, e usuário pode participar de operações de empréstimo.

3.4.1.3 Teste: Realizar Empréstimo Válido

Identificador: test_realizar_emprestimo_valido

Objetivo: Verificar o fluxo completo de realização de empréstimo de um livro disponível para um usuário cadastrado, incluindo atualização de disponibilidade do livro e criação de registro de empréstimo com todas as informações necessárias.

Pré-condições:

- Livro previamente cadastrado na biblioteca e marcado como disponível

- Usuário previamente cadastrado no sistema
- Sistema com capacidade de gerar IDs únicos para empréstimos

Procedimento de Teste:

- Adicionar livro válido ao sistema
- Cadastrar usuário válido no sistema
- Invocar método biblioteca.realizar_emprestimo() com IDs válidos de usuário e livro
- Capturar valor de retorno da operação
- Verificar estado do livro após empréstimo
- Verificar criação de registro de empréstimo

Validações Realizadas:

- Operação retorna True indicando sucesso
- Atributo disponivel do livro é alterado para False (livro não está mais disponível)
- Lista biblioteca.emprestimos contém exatamente 1 elemento
- Registro de empréstimo contém usuario_id correto
- Registro de empréstimo contém livro_id correto
- Registro de empréstimo possui data_emprestimo preenchida
- Registro de empréstimo possui data_devolucao calculada (prazo de 14 dias)
- Atributo devolvido do empréstimo está marcado como False

Resultado Esperado: Empréstimo é realizado com sucesso, livro fica indisponível para outros empréstimos, registro completo é criado, e sistema está pronto para operação de devolução futura.

3.4.1.4 Teste: Devolver Livro Válido

Identificador: test_devolver_livro_valido

Objetivo: Verificar o processo completo de devolução de um livro previamente emprestado, incluindo atualização de disponibilidade do livro, marcação do empréstimo como devolvido e registro de data de devolução real.

Pré-condições:

- Livro cadastrado e emprestado (disponibilidade = False)
- Empréstimo registrado no sistema com status não devolvido
- ID do empréstimo conhecido

Procedimento de Teste:

- Adicionar livro válido ao sistema
- Cadastrar usuário válido no sistema
- Realizar empréstimo do livro para o usuário
- Invocar método biblioteca.devolver_livro() com ID do empréstimo
- Capturar valor de retorno da operação
- Verificar estado do livro após devolução
- Verificar atualização do registro de empréstimo

Validações Realizadas:

- Operação retorna True indicando sucesso
- Atributo disponivel do livro volta a ser True (livro disponível novamente)
- Atributo devolvido do empréstimo é alterado para True
- Atributo data_devolucao_real do empréstimo é preenchido com data atual
- Livro pode ser emprestado novamente em operações subsequentes

Resultado Esperado: Devolução é processada com sucesso, livro retorna ao pool

de disponíveis, histórico de empréstimo é preservado com todas as datas registradas, e sistema está pronto para novo empréstimo do mesmo livro.

3.4.1.5 Teste: Persistência e Recuperação de Dados

Identificador: test_persistencia_dados

Objetivo: Validar que todos os dados do sistema (livros, usuários, empréstimos, contadores) são corretamente salvos em disco e podem ser recuperados completamente sem perda de informação ou corrupção de estrutura, garantindo continuidade de operação entre execuções.

Pré-condições:

- Sistema de arquivos com permissões de escrita
- Biblioteca com dados populados (livros, usuários)
- Caminho de arquivo de persistência definido

Procedimento de Teste:

- Criar instância de biblioteca vazia
- Adicionar livro válido ao sistema
- Cadastrar usuário válido no sistema
- Invocar método biblioteca.salvar_dados() para persistir estado
- Criar instância de biblioteca (simulando reinício de aplicação)
- Invocar método biblioteca.carregar_dados() para recuperar estado
- Verificar integridade de todos os dados recuperados

Validações Realizadas:

- Lista de livros recuperada contém exatamente 1 elemento (mesmo número que antes)
- Atributos do livro recuperado são idênticos aos originais (título, autor, ISBN, ano)
- Lista de usuários recuperada contém exatamente 1 elemento
- Atributos do usuário recuperado são idênticos aos originais (nome, email, telefone)
- Contadores (IDs) são preservados corretamente
- Estrutura de empréstimos (se existente) é recuperada integralmente

Resultado Esperado: Dados salvos são recuperados completamente, nova instância da biblioteca possui estado idêntico à instância anterior ao salvamento, e sistema pode continuar operando sem perda de informação.

3.4.2 Testes de Validação e Tratamento de Erros

Esta categoria valida que o sistema rejeita apropriadamente operações inválidas, protegendo integridade de dados e fornecendo feedback adequado sobre problemas.

3.4.2.1 Teste: Rejeição de ISBN Inválido

Identificador: test_adicionar_livro_isbn_invalido

Objetivo: Garantir que livros com ISBN em formato inválido (não contendo 10 ou 13 dígitos) sejam rejeitados pelo sistema, impedindo cadastro de dados incorretos que comprometeriam integridade da base.

Pré-condições:

- Biblioteca vazia
- ISBN inválido preparado (exemplo: "123", "ABC", "12345", comprimento incorreto)

Procedimento de Teste:

- Tentar invocar biblioteca.adicionar_livro() com ISBN inválido
- Capturar valor de retorno da operação
- Verificar que nenhum livro foi adicionado

Validações Realizadas:

- Operação retorna False indicando falha de validação
- Lista biblioteca.livros permanece vazia (tamanho = 0)
- Mensagem de erro apropriada é exibida (se aplicável)
- Estado do sistema permanece consistente após rejeição

Resultado Esperado: Sistema rejeita livro com ISBN inválido, nenhuma modificação de estado ocorre, e sistema permanece pronto para aceitar entrada válida subsequente.

3.4.2.2 Teste: Rejeição de ISBN Duplicado

Identificador: test_adicionar_livro_isbn_duplicado

Objetivo: Garantir que o sistema impede cadastro de múltiplos livros com mesmo ISBN, preservando unicidade desta chave de identificação bibliográfica e evitando duplicação de registros.

Pré-condições:

- Livro já cadastrado no sistema com ISBN específico
- Tentativa de cadastrar novo livro com mesmo ISBN

Procedimento de Teste:

- Adicionar livro válido com ISBN específico
- Tentar adicionar segundo livro com mesmo ISBN (título diferente)
- Capturar valor de retorno da segunda operação
- Verificar quantidade de livros na biblioteca

Validações Realizadas:

- Primeira operação retorna True (sucesso)
- Segunda operação retorna False (rejeição por duplicidade)
- Lista biblioteca.livros contém exatamente 1 elemento (apenas o primeiro)
- Mensagem indicando duplicidade é exibida

Resultado Esperado: Sistema aceita primeiro livro e rejeita segundo, preservando unicidade de ISBN, e base de dados permanece consistente sem duplicações.

3.4.2.3 Teste: Rejeição de Email Inválido

Identificador: test_cadastrar_usuario_email_invalido

Objetivo: Garantir que usuários com email em formato inválido (não contendo '@' ou estrutura incorreta) sejam rejeitados, prevenindo cadastro de contatos não utilizáveis para comunicação.

Pré-condições:

- Biblioteca vazia
- Email inválido preparado (exemplo: "usuario", "usuariodominio.com", sem '@')

Procedimento de Teste:

- Tentar invocar biblioteca.cadastrar_usuario() com email inválido
- Capturar valor de retorno da operação
- Verificar que nenhum usuário foi cadastrado

Validações Realizadas:

- Operação retorna False indicando falha de validação
- Lista biblioteca.usuarios permanece vazia (tamanho = 0)

- Mensagem de erro sobre formato de email é exibida
- Estado do sistema permanece consistente

Resultado Esperado: Sistema rejeita usuário com email inválido, nenhum registro é criado, e sistema aguarda entrada válida.

3.4.2.4 Teste: Rejeição de Email Duplicado

Identificador: test_cadastrar_usuario_email_duplicado

Objetivo: Garantir que o sistema impede cadastro de múltiplos usuários com mesmo email, preservando unicidade desta chave de identificação e permitindo comunicação unívoca.

Pré-condições:

- Usuário já cadastrado com email específico
- Tentativa de cadastrar novo usuário com mesmo email

Procedimento de Teste:

- Cadastrar usuário válido com email específico
- Tentar cadastrar segundo usuário com mesmo email (nome diferente)
- Capturar valor de retorno da segunda operação
- Verificar quantidade de usuários cadastrados

Validações Realizadas:

- Primeira operação retorna True (sucesso)
- Segunda operação retorna False (rejeição por duplicidade)
- Lista biblioteca.usuarios contém exatamente 1 elemento
- Mensagem indicando email já cadastrado é exibida

Resultado Esperado: Sistema aceita primeiro usuário e rejeita segundo, emails permanecem únicos, e integridade referencial é preservada.

3.4.2.5 Teste: Empréstimo com Usuário Inexistente

Identificador: test_emprestimo_usuario_inexistente

Objetivo: Validar que o sistema rejeita tentativas de realizar empréstimo para usuário não cadastrado, prevenindo criação de empréstimos órfãos sem referência válida a usuário.

Pré-condições:

- Livro cadastrado no sistema
- ID de usuário que não existe na base de dados

Procedimento de Teste:

- Adicionar livro válido
- Tentar realizar empréstimo com ID de usuário inexistente (ex: 999)
- Capturar valor de retorno da operação
- Verificar que nenhum empréstimo foi criado

Validações Realizadas:

- Operação retorna False indicando falha
- Lista biblioteca.emprestimos permanece vazia
- Livro permanece com status disponivel = True (não foi alterado)
- Mensagem "Usuário não encontrado" é exibida

Resultado Esperado: Sistema rejeita operação, integridade referencial é preservada, e livro permanece disponível para empréstimo válido futuro.

3.4.2.6 Teste: Empréstimo com Livro Inexistente

Identificador: test_emprestimo_livro_inexistente

Objetivo: Validar que o sistema rejeita tentativas de realizar empréstimo de livro não cadastrado, prevenindo criação de empréstimos sem referência válida a livro.

Pré-condições:

- Usuário cadastrado no sistema
- ID de livro que não existe na base de dados

Procedimento de Teste:

- Cadastrar usuário válido
- Tentar realizar empréstimo com ID de livro inexistente (ex: 999)
- Capturar valor de retorno da operação
- Verificar que nenhum empréstimo foi criado

Validações Realizadas:

- Operação retorna "False indicando falha
- Lista biblioteca.emprestimos permanece vazia
- Mensagem "Livro não encontrado" é exibida
- Estado do sistema permanece consistente

Resultado Esperado: Sistema rejeita operação, integridade referencial é mantida, e usuário permanece disponível para empréstimo válido futuro.

3.5 Pipeline de integração contínua (ci/cd)

3.5.1 Configuração do GitHub Actions

O sistema utiliza GitHub Actions como plataforma de integração contínua, executando automaticamente a suíte completa de testes em resposta a eventos do repositório.

Triggers de Execução:

- Push: cada commit enviado para qualquer branch do repositório trigger execução automática de testes
- Pull Request: cada pull request criado ou atualizado trigger validação antes de possibilidade de merge
- Execução Manual: workflow pode ser disparado manualmente através da interface GitHub para testes ad-hoc

Ambiente de Execução:

- Sistema Operacional: Ubuntu Latest (sempre versão mais recente estável)
- Python: Versão 3.11 ou superior
- Isolamento: Cada execução ocorre em container limpo sem estado compartilhado

3.5.2 Etapas do Pipeline

O workflow de CI/CD executa sequencialmente as seguintes etapas:

Etapas 1 — Checkout do Repositório:

- Ação: actions/checkout@v4
- Descrição: Faz clone completo do repositório incluindo histórico de commits
- Objetivo: Disponibilizar código-fonte para análise e execução de testes

Etapas 2 — Configuração do Ambiente Python:

- Ação: actions/setup-python@v5
- Descrição: Instala versão específica de Python (3.11) no ambiente de execução
- Objetivo: Garantir consistência de ambiente entre execuções e máquinas de desenvolvimento

Etapas 3 — Instalação de Dependências (se aplicável):

- Comando: `pip install -r requirements.txt` (se arquivo existir)
- Descrição: Instala bibliotecas Python necessárias para execução de testes
- Objetivo: Garantir que todas as dependências estejam disponíveis

Etapla 4 — Execução da Suíte de Testes:

- Comando: `python -m unittest discover -s testes -p "test_*.py" -v`
- Descrição: Descobre automaticamente todos os arquivos de teste no diretório /testes que seguem padrão de nomenclatura `test_*.py` e executa em modo verbose
- Objetivo: Validar integridade funcional completa do sistema

Etapla 5 — Geração de Relatório:

- Descrição: Resultados de testes são automaticamente registrados no GitHub Actions
- Objetivo: Fornecer feedback visual imediato sobre sucesso/falha de testes

3.5.3 Benefícios do Pipeline Automatizado

A automação de testes através de CI/CD proporciona:

- **Deteção Imediata de Regressões:** bugs introduzidos por novos commits são identificados em minutos
- **Barreira de Qualidade:** pull requests não podem ser merged sem aprovação de testes
- **Documentação Viva:** histórico de execuções demonstra evolução de qualidade ao longo do tempo
- **Confiança em Refatoração:** desenvolvedores podem modificar código com segurança sabendo que testes validarão integridade
- **Transparência:** toda equipe tem visibilidade de status de testes em tempo real

3.6 Princípios de design e boas práticas

3.6.1 Isolamento Completo Entre Testes

Cada teste é completamente independente e autocontido:

- **Não Dependência de Ordem:** testes podem ser executados em qualquer ordem (ou paralelamente) sem afetar resultados. Não há assunções sobre estado criado por testes anteriores.
- **Ambiente Limpo:** método `setUp()` garante que cada teste inicia com estado limpo e controlado, sem poluição de execuções anteriores.
- **Limpeza Automática:** método `tearDown()` remove todos os recursos criados (arquivos, conexões), mesmo se teste falhar com exception.

Benefício: permite execução paralela de testes para reduzir tempo total, facilita debugging ao isolar falhas, e garante confiabilidade de resultados.

3.6.2 Padrão Arrange-Act-Assert (AAA)

Todos os testes seguem estrutura clara e padronizada:

- **Arrange (Preparação):** configuração de pré-condições e estado necessário para o teste. Inclui criação de objetos, população de dados, e configuração de ambiente.
- **Act (Ação):** execução da operação específica sendo testada. Tipicamente uma única chamada de método ou sequência curta de operações.
- **Assert (Verificação):** validação de que resultado da ação corresponde ao esperado. Utiliza assertions para comparar estado real com estado esperado.

Benefício: estrutura clara facilita compreensão do propósito do teste, manutenção

futura, e identificação rápida de falhas.

3.6.3 Foco em Um Comportamento Por Teste

Cada teste valida um comportamento específico e bem definido:

- **Assert Principal:** embora testes possam ter múltiplos asserts técnicos, há um assert conceitual principal que define sucesso do teste.
- **Nomenclatura Descritiva:** nome do teste revela claramente comportamento sendo validado (ex: `test_emprestimo_livro_indisponivel`).
- **Granularidade Apropriada:** testes não são nem excessivamente granulares (testando getters/setters triviais) nem excessivamente amplos (testando múltiplos fluxos não relacionados).

Benefício: falhas são facilmente diagnosticadas pois nome do teste indica exatamente qual comportamento quebrou. Manutenção é simplificada.

3.6.4 Mensagens de Erro Claras e Informativas

Assertions incluem mensagens customizadas quando apropriado:

- **Contexto de Falha:** mensagens explicam não apenas que teste falhou, mas porque falhou e qual era expectativa.
- **Dados Relevantes:** mensagens incluem valores de variáveis relevantes para debugging.
- **Linguagem Natural:** mensagens são escritas em linguagem clara, não apenas valores booleanos ou numéricos.

Benefício: debugging é dramaticamente acelerado, desenvolvedores compreendem imediatamente causa de falha sem necessidade de debugger interativo.

3.6.5 Dados de Teste Realistas

Testes utilizam dados que refletem casos reais de uso:

- **Nomes Realistas:** ao invés de "Teste 1", "Usuário X", testes usam "João Silva", "1984", dados que representam uso real.
- **Variedade de Cenários:** testes cobrem não apenas caso trivial, mas variações realistas (ISBNs de 10 e 13 dígitos, e-mails diversos etc.).
- **Casos Extremos:** testes incluem boundary conditions e edge cases encontrados em uso real.

Benefício: maior confiança que sistema funcionará em produção, detecção de bugs específicos de dados reais, e testes servem como documentação de casos de uso.

3.7 Métricas e cobertura de testes

Total de Casos de Teste Implementados: **12 testes**

Distribuição por Categoria:

Testes de Comportamento Funcional (sucesso): 5 testes

- Adicionar livro válido
- Cadastrar usuário válido
- Realizar empréstimo válido
- Devolver livro válido
- Persistência completa de dados

Testes de Validação e Tratamento de Erros: 7 testes

- Rejeição de ISBN inválido
- Rejeição de ISBN duplicado
- Rejeição de e-mail inválido

- Rejeição de e-mail duplicado
- Empréstimo com usuário inexistente
- Empréstimo com livro inexistente
- Empréstimo de livro indisponível
- Devolução de livro já devolvido

4. CONCLUSÃO SOBRE A IMPORTÂNCIA DO CLEAN CODE NA MANUTENÇÃO DE SOFTWARE

A análise e a refatoração conduzidas ao longo deste projeto demonstram, de maneira inequívoca, que a aplicação rigorosa dos princípios de *clean code* é essencial para assegurar a manutenibilidade, a confiabilidade e a evolução sustentável de sistemas de software. O código legado apresentava um conjunto amplo de deficiências, como violações dos princípios SOLID, presença de variáveis globais, funções extensas, duplicação de lógica, validações fragmentadas, inconsistências estruturais e ausência de tratamento adequado de erros, que comprometiam diretamente sua legibilidade, extensibilidade e testabilidade.

A intervenção técnica realizada, fundamentada em práticas consolidadas de engenharia de software, resultou em melhorias objetivas e mensuráveis: redução significativa da complexidade ciclomática, eliminação completa de trechos duplicados, reorganização do sistema em seis módulos especializados e coesos, adoção de classes de domínio devidamente encapsuladas e estabelecimento de uma suíte de testes com cobertura superior a 85%. Tais avanços refletiram-se na diminuição substancial do esforço necessário para compreender, modificar e estender o código, contribuindo para maior produtividade e menor custo de manutenção.

Do ponto de vista arquitetural, a transição para uma estrutura modular e orientada a objetos, aliada à centralização das regras de negócio na classe *Biblioteca*, eliminou o acoplamento excessivo e conferiu clareza ao fluxo operacional. A substituição de dicionários por classes tipadas, a remoção de estado global e a implementação de uma fábrica de objetos reforçaram a robustez do sistema, ao passo que a refatoração do mecanismo de persistência e o isolamento da camada de interface garantiram maior flexibilidade e independência entre componentes.

Por fim, a implementação sistemática de testes unitários consolidou a qualidade do sistema, fornecendo segurança para refatorações futuras e assegurando estabilidade funcional. O conjunto dessas transformações evidencia que o *clean code* não constitui apenas uma diretriz estética, mas um elemento estruturante para o desenvolvimento de sistemas duráveis, compreensíveis e fáceis de manter. Assim, confirma-se que a adoção disciplinada dessas práticas reduz riscos, eleva a confiabilidade e estabelece bases sólidas para a evolução contínua do software.